Class 13- Coloured CNNs

- Image classification: filters passing by and learning patterns. Universal to spatial imagery.
- Vs. images in corners not centered to the middle of the filter. Padding adding zeros in corners for better retention.
- Padding is a parameter that is used inside the convolutional layer to add layers of zeros to our input images.
- 255 pixels for normal RGBs -> normalisation
- Pooling: reducing space by extracting features
- Flatten layer: 1D vector. Converts everything into 1D layer
- Convolution + dense layers to classify
- Learning rate, batch size, optimizer

Padding Techniques in Convolutional Neural Networks (CNN)

Padding is a technique used in Convolutional Neural Networks to control how the borders of an image are handled during the convolution operation. There are several approaches to applying padding, each with a specific effect on the model's outputs.

Types of Padding

1. **Padding = 'valid'**

- **Description**: No padding (zeros) is added to the edges of the image.
- **Effect**: The output of the convolution is **smaller** than the input, as the operation is performed only where the filter fits completely within the image.

2. **Padding = 'same'**

- **Description**: The image is padded with zeros along the borders to ensure that the **output dimension is equal to the input dimension**.
- **Effect**: The number of border pixels is adjusted to ensure that the output has the same size as the input.

3. **Custom Padding (Arbitrary)**

- **Description**: You can explicitly specify how many zeros to add around the image.
- **Effect**: Provides greater control over the padding amount, allowing for symmetric or asymmetric padding.

4. **Reflective Padding**

- **Description**: The borders of the image are padded with a reflection of the pixel values at the borders, instead of zeros.
- **Effect**: Useful when there is significant information in the image borders, as it preserves characteristics without introducing artificial patterns.

5. **Asymmetric Padding**

- **Description**: Different amounts of padding are applied to each side of the image (non-uniform).
- **Effect**: Useful when you want to focus on a specific part of the image (e.g., more padding on the left or top).

6. **Circular Padding**

- **Description**: Also known as "wrap-around padding", this type of padding involves "wrapping" the image, where the borders connect.
- **Effect**: Useful for tasks involving repetitive patterns, such as satellite images or continuous textures.

7. **Padding with Dynamic Data**

- **Description**: Padding that depends on variables from the input or contextual features.
- **Effect**: Allows the model to adapt to different types of data dynamically, adjusting padding as needed.

8. **Slicing Instead of Padding**

- **Description**: Instead of adding padding, parts of the image are sliced or cropped. This can be useful to focus on the central region of the image.
- **Effect**: Useful in tasks like edge detection or when you want to reduce the borders.

Conclusion

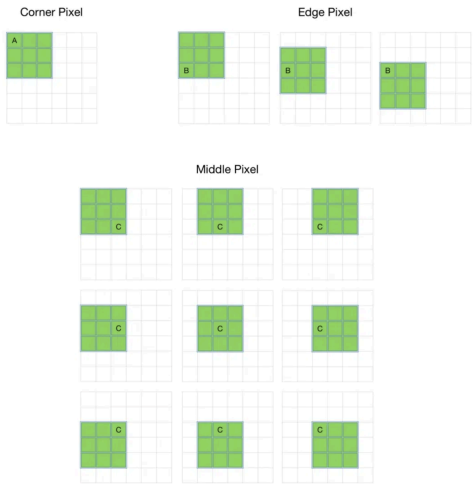
Each padding technique has its advantages depending on the type of data and the task at hand. **'Same' padding** is the most common when you want to maintain the image dimensions, while other techniques like **Reflective Padding** or **Circular Padding** may be more suitable for specific scenarios.

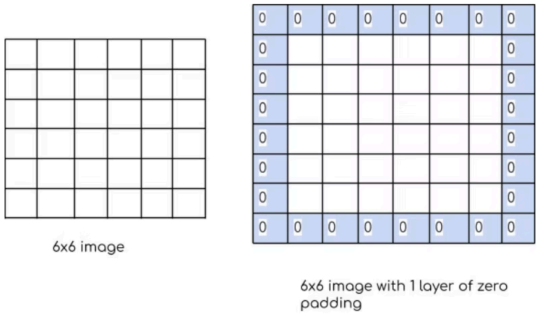
Architecture

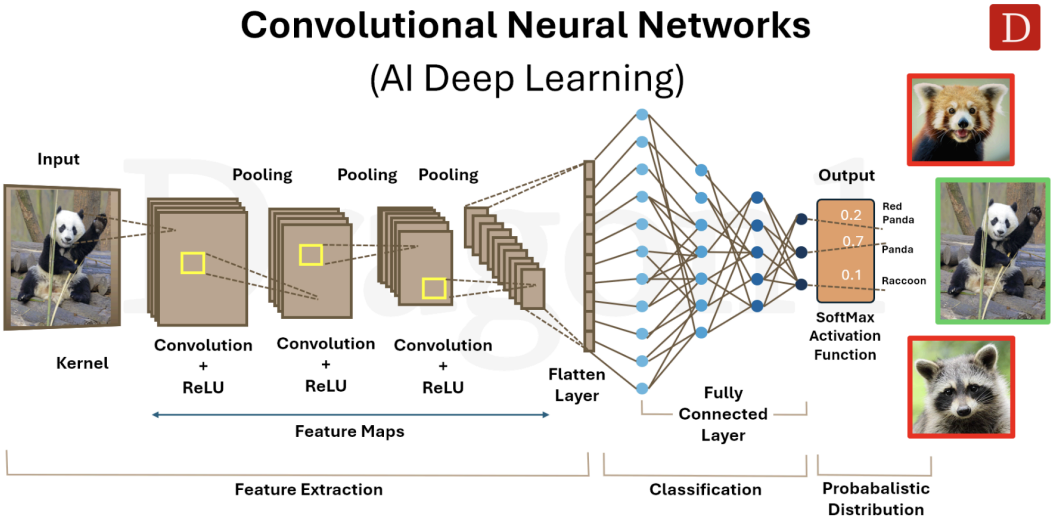
```
# Initializing the Input
input = tf.constant([[1, 2, 5], [3, 4, 6]])
padding = tf.constant([[1, 1], [2, 2]])
res = tf.pad(input, padding, mode ='REFLECT')


model = Sequential()
model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=(32, 32, 3)))

model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# ADD more layers to your model compile and visualize the results
model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(filters=128, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'],
             )
history = model.fit(X_train,
                    y_train,
                    batch_size=32,
                    epochs=20,
                    verbose=1,
                    validation_split=0.3,
                   )
```







For example, for an (8 x 8) image and (3 x 3) filter, the output resulting after the convolution operation would be of size (6 x 6). Thus, the image shrinks every time a convolution operation is performed. This places an upper limit to the number of times such an operation could be performed before the image reduces to nothing thereby precluding us from building deeper networks.Also, the pixels on the corners and the edges are used much less than those in the middle.

❗ Formula for Output Size Calculation in Convolutional Layer with Padding

The general formula for calculating the output size of a convolutional layer with padding is:

$$\text{Output Size} = \frac{\text{Input Size} - \text{Kernel Size} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

- Conv2D: Convolutional layer. The convolutional layer applies filters over the input image (or the output from previous layers) to extract features like edges, textures, and patterns.
    - filters=32: Defines the number of filters (or kernels) applied. In this case, 32 convolutional filters.
    - kernel_size=3: The size of the filter (3x3).
    - padding='same': The image is padded with zeros along the borders so that the output of the convolution has the same size as the input.
    - activation='relu': The activation function is ReLU (Rectified Linear Unit). It introduces non-linearity and helps to solve the vanishing gradient problem in deep networks.
- MaxPooling2D: Pooling layer. After applying the convolution, pooling reduces the dimensionality of the output (compressing the image) while preserving the most important features.
    - pool_size=(2, 2): Pooling is performed with a 2x2 window, reducing the image size by half in each direction.

```
model = Sequential()
```

- **Sequential**: Creates a sequential model, meaning a stack of layers where the output of one layer becomes the input of the next. This type of model is useful when you have a simple network architecture, where the layers are stacked in sequence.

Convolutional and Pooling Layers:

```
model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

- **Conv2D**: Convolutional layer. The convolutional layer applies filters over the input image (or the output from previous layers) to extract features like edges, textures, and patterns.
  - **filters=32**: Defines the number of filters (or kernels) applied. In this case, 32 convolutional filters.
  - **kernel_size=3**: The size of the filter (3x3).
  - **padding='same'**: The image is padded with zeros along the borders so that the output of the convolution has the same size as the input.
  - **activation='relu'**: The activation function is **ReLU** (Rectified Linear Unit). It introduces non-linearity and helps to solve the vanishing gradient problem in deep networks.
- **MaxPooling2D**: Pooling layer. After applying the convolution, pooling reduces the dimensionality of the output (compressing the image) while preserving the most important features.
  - **pool_size=(2, 2)**: Pooling is performed with a 2x2 window, reducing the image size by half in each direction.

```
model.compile()
```

```
Start coding or generate with AI.
```

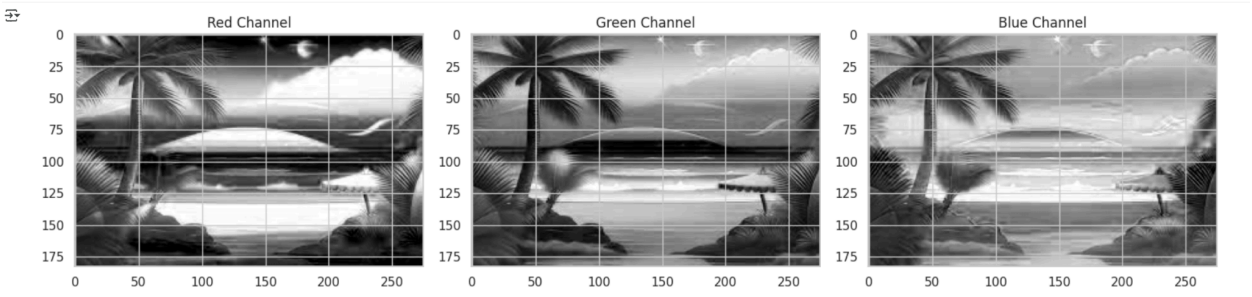
```
model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'],
             )
```

1. optimizer='rmsprop':
   - RMSprop is an optimizer that adjusts the learning rate for each parameter based on the average of recent gradients. It is widely used for training deep learning models.
   - It helps stabilize the learning process by making the training more efficient, especially when dealing with noisy gradients or sparse data.
2. loss='categorical_crossentropy':
   - Categorical Crossentropy is the loss function used for multi-class classification problems. It calculates the difference between the true labels and the predicted probabilities (from the softmax activation in the last layer).
   - This function penalizes incorrect classifications, and the goal during training is to minimize this loss, i.e., make the predicted probabilities as close as possible to the actual class labels.
3. metrics=['accuracy']:
   - Accuracy is a metric used to evaluate how well the model is performing. It calculates the percentage of correct predictions out of the total number of predictions.
   - During training, the accuracy metric is calculated after each epoch, helping you track the model's performance on the training and validation sets.

Class 14- Image Transformation of Images

- Grey scale images for normalization

```
blue, green, red = cv2.split(input_image)
# Create a figure with three subplots
fig, axs = plt.subplots(1, 3)
fig.set_size_inches(15,15)

# Display the red channel in the first subplot
axs[0].imshow(red, cmap='gray')
axs[0].set_title('Red Channel')

# Display the green channel in the second subplot
axs[1].imshow(green, cmap='gray')
axs[1].set_title('Green Channel')

# Display the blue channel in the third subplot
axs[2].imshow(blue, cmap='gray')
axs[2].set_title('Blue Channel')

# Adjust the layout of the subplots
fig.tight_layout()

# Show the figure
plt.show()# Why does it seems like a gray scale image ?
```



-

Applying Laplasian Derivative on the grayscale image

- The Laplacian of an image highlights regions of rapid intensity change and it's widely used as a prior step for edge detection. The operator normally takes a single grayscale image as input and generates another grayscale image as output

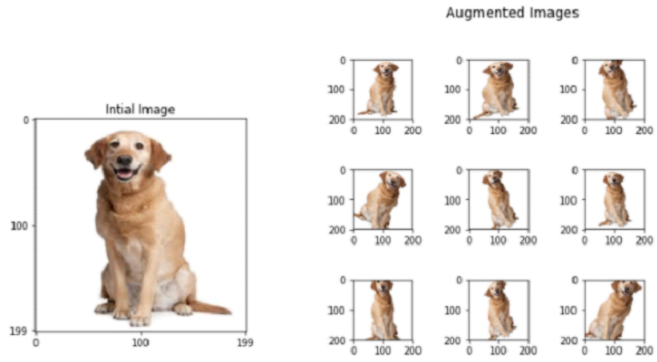
Image preprocessing steps

- Flowers into arrays and resize them
- Resize to standard size to a specific shape
- Pixelization for array manipulation

Sparse categorical cross entropy = 0,1,2,3
Categorical cross entropy= 0,0,1,0

- Datagen flow

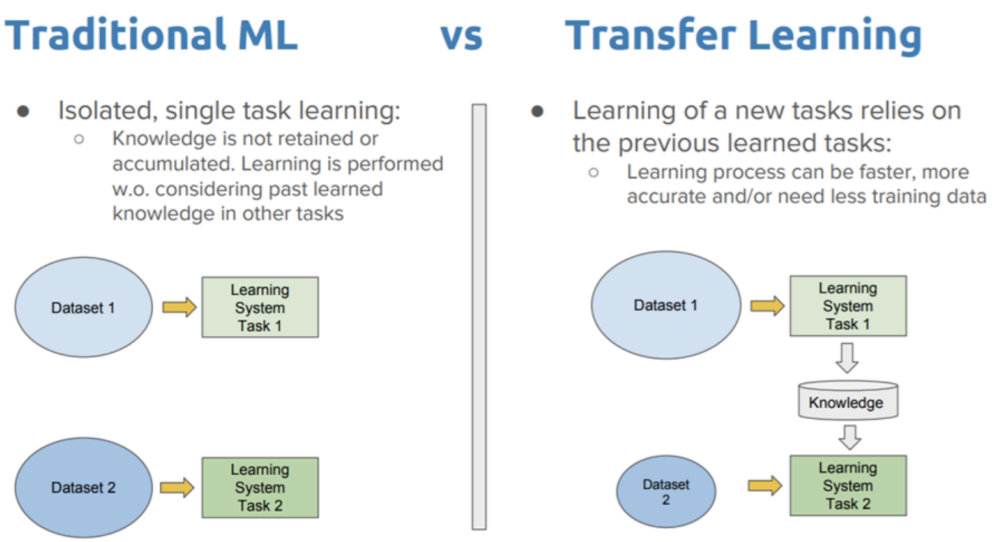
Image Data Generator is a method from Keras that is widely used to avoid overfitting in the training process with the images. It has a wide range of parameters to generate new images during the training proces.




```
datagen = ImageDataGenerator(
          featurewise_center=False,  # set input mean to 0 over the dataset
          samplewise_center=False,  # set each sample mean to 0
          featurewise_std_normalization=False,  # divide inputs by std of the dataset
          samplewise_std_normalization=False,  # divide each input by its std
          zca_whitening=False,  # apply ZCA whitening
          rotation_range=10,  # randomly rotate images in the range (degrees, 0 to 180)
          zoom_range = 0.1, # Randomly zoom image
          width_shift_range=0.2,  # randomly shift images horizontally (fraction of total width)
          height_shift_range=0.2,  # randomly shift images vertically (fraction of total height)
          horizontal_flip=True,  # randomly flip images
          vertical_flip=True)  # randomly flip images


datagen.fit(x_train)
```

Class 16- Transfer Learning

1st layers= bigger layers
Last layers: More specification



Assuming you have 1000 images of cats and dogs, and you want to build a classifier for this binary class problem. It is possible to train the model from scratch however you have a small ammount of data and there is a high probability that you would not create a perfect architechture from the first attempts. Even if you have vast ammount of data, perfect architechture of the CNN, you still would need some days to train the model on a relativelly accessible GPU.

In that case it is better to use Transfer Learning models and adapt the task in hand.

+ Code + Text

How? (The big question)

1. Chose the pre-trained model to work with.
2. Create your base model (pre-trained).
3. Freeze the layers of the pre-trained model and add yours.
4. Train new layers on the new dataset or task.
5. (Optional) Improve the model via fine-tunning.